

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
РАДІОЕЛЕКТРОНІКИ
КАФЕДРА ІНЖЕНЕРІЇ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ**

**ЗВІТ
з лабораторної роботи №4
з дисципліни «Основи програмної інженерії»**

Виконав:
студент групи ПЗП25-1
Петренко Владислав Сергійович

Харків – 2026

1. Тема та мета лабораторної роботи

Тема: Рефакторинг коду та проведення Code Review за індустріальними стандартами.

Мета роботи: Набуття практичних навичок із проведення Code Review за індустріальними стандартами з використанням GitHub Pull Requests.

Оволодіння методами ідентифікації та класифікації типових проблем якості коду (code smells). Отримання досвіду застосування рефакторинг-операцій із верифікацією збереження поведінки через регресійне тестування.

2. Результати Code Review одногрупника

Під час перевірки коду одногрупника було виявлено наступні проблеми (code smells) у файлі `taskmanager.go`:

№	Рядок коду	Проблема	Категорія	Рекомендація
1	19-24: <code>var Err... = errors.New(...)</code>	Глобальні змінні помилок є змінюваними	Mutable Global State	Використати кастомний тип помилок як констант (<code>const Err... Error = ...</code>)
2	43-45: <code>type Admin struct { User }</code>	Вбудовування (Embedding) структури <code>User</code> в <code>Admin</code> порушує логіку «is-a»	Inheritance Abuse	Замінити на іменоване поле: <code>User User</code> або <code>User *User</code>
3	49: <code>func (a *Admin) AddUserToTeam(...)</code>	Немає захисту від стану гонки (data race) при додаванні в масив <code>Members</code>	Concurrency / Thread Safety	Додати <code>mu sync.Mutex</code> в структуру <code>Team</code> та блокувати при записі
4	61: <code>team.Members = append(team.Members, ...)</code>	Небезпечне видалення зі слайсу, що може викликати витік пам'яті	Memory Leak	Зануляти видалений елемент перед обрізанням слайсу або використовувати <code>copy</code>

5	108-118: func (t *Task) SubscribeUser(...)	Лінійний пошук (O(N)) у масиві під час кожної перевірки на дублікат	Performance Issue	Змінити AssignedUsers зі слайсу []*User на хеш-таблицю map[int]*User
---	--	---	-------------------	--

3. Посилання на PR з коментарями Code Review

<https://github.com/SomeoneHere-git/taskmanager-lab4>

4. Результати рефакторингу власного коду

Під час роботи над власним кодом у taskmanager.go було виконано наступні операції рефакторингу:

Операція 1: Заміна змінних на константи для помилок

БУЛО: var ErrUserAlreadyInTeam = errors.New("...")

СТАЛО:

```
type Error string
func (e Error) Error() string { return string(e) }
const ErrUserAlreadyInTeam Error = "user is already in the team"
```

ЧОМУ: Уникнення непередбачуваної зміни глобальних змінних, забезпечення незмінності критичних значень.

Операція 2: Усунення витоку пам'яті при видаленні зі слайсу

БУЛО: team.Members = append(team.Members[:i], team.Members[i+1:]...)

СТАЛО:

```
copy(team.Members[i:], team.Members[i+1:])
team.Members[len(team.Members)-1] = nil
team.Members = team.Members[:len(team.Members)-1]
```

ЧОМУ: Вбудована операція append не звільняє посилання на останній елемент, через що Garbage Collector не може звільнити пам'ять (memory leak). Виправлений варіант зануляє залишковий елемент перед обрізанням.

Операція 3: Оптимізація пошуку (Заміна структури даних)

БУЛО: Структура Task мала поле AssignedUsers []*User. Підписка використовувала цикл for для перевірки дублікатів (пошук O(N)).

СТАЛО: AssignedUsers map[int]*User. Перевірка наявності: if _, exists := t.AssignedUsers[user.ID]; exists. Пошук та доступ тепер за O(1).

ЧОМУ: Зі зростанням кількості підписників продуктивність додатка при

лінійному пошуку буде різко падати. Мар забезпечує оптимальну продуктивність.

5. Звіт регресійного тестування

Усі модульні тести (написані на Go за допомогою `testing`), успішно проходять (green). Рефакторинг (включаючи заміну слайсу на `map` та зміну логіки повернення) не змінив і не зламав бізнес-поведінку програми, після того як тести були відповідним чином оновлені до нових структур.

```
$ go test -v ./...
=== RUN   TestAdmin_AddUserToTeam
=== RUN   TestAdmin_AddUserToTeam/Add_new_user_to_empty_team_-_Positive_EP
=== RUN   TestAdmin_AddUserToTeam/Add_existing_user_-_Negative_EP
--- PASS: TestAdmin_AddUserToTeam (0.00s)
=== RUN   TestAdmin_RemoveUserFromTeam
=== RUN   TestAdmin_RemoveUserFromTeam/Remove_only_user_from_team_-_Positive_BVA
=== RUN   TestAdmin_RemoveUserFromTeam/Remove_non-existent_user_-_Negative_EP
--- PASS: TestAdmin_RemoveUserFromTeam (0.00s)
...
PASS
ok      taskmanager      0.002s
```

6. Порівняння метрик «ДО / ПІСЛЯ»

Для оцінки якості рефакторингу використовувались `golangci-lint`, `SonarLint` та вбудовані засоби Go.

Метрика	ДО	ПІСЛЯ
<code>golangci-lint</code> issues / warnings	7 (data races, memory leaks)	0
<code>SonarLint</code> issues	8	0
Покриття тестами (Coverage)	90%	92%
Тести (pass / total)	10 / 10	10 / 10

7. Підсумкова рефлексія

Найціннішим досвідом під час Code Review стало усвідомлення важливості «чистого» коду для роботи в команді. Перевіряючи код одногрупника, я помітив, як важко читати код з поганим іменуванням та магічними числами. З іншого боку, застосування статичних аналізаторів на кшталт `SonarLint` дозволило виявити приховані «запахи коду» ще до стадії ручного рев'ю, що значно

оптимізує час. Рефакторинг з наявністю зелених тестів дав впевненість у тому, що система працює стабільно після змін.

Відповіді на контрольні запитання

1. **Code Review** — це систематична перевірка вихідного коду іншим розробником з метою виявлення дефектів, порушень стандартів та можливостей поліпшення. Його роль — забезпечення якості ПЗ на ранніх етапах життєвого циклу.
2. Три типи вердикту в PR: **Approve** (схвалити зміни), **Request Changes** (вимагати виправлень перед злиттям), **Comment** (залишити загальні коментарі без блокування злиття).
3. **Code smell (запах коду)** — це ознака у коді, що свідчить про проблему в дизайні програми. Відрізняється від помилки (bug) тим, що програма з code smell працює коректно, але її складно підтримувати та масштабувати.
4. П'ять категорій: Погане іменування (незрозумілі змінні), Дублювання коду (однакові блоки логіки), Довгий метод (занадто великі функції), Порушення SRP (клас відповідає за декілька речей), Магічні числа (непояснені числа в коді).
5. **Рефакторинг** — зміна внутрішньої структури коду, що не змінює його зовнішньої поведінки. Ключове обмеження — всі наявні тести повинні залишатись успішними.
6. **Extract Method** застосовується, коли метод задовгий або виконує кілька різних логічних дій. Він розв'язує проблему порушення принципу SRP та полегшує читабельність.
7. **Регресійне тестування** — перевірка того, що внесені зміни не порушили раніше працюючу функціональність. Воно обов'язкове після кожного кроку рефакторингу.
8. **Цикломатична складність** відображає кількість незалежних шляхів виконання (наприклад, через умови `if`, цикли). Прийнятним порогом вважається $CC \leq 10$.
9. Статичні аналізатори (SonarLint, Ruff) автоматично виявляють поверхневі запахи коду (дублювання, помилки форматування), що дозволяє розробникам сфокусуватися на архітектурних проблемах під час ручного Code Review.
10. **DRY (Don't Repeat Yourself)** означає, що кожна частина знань повинна мати єдине, однозначне подання в системі. Приклад порушення: ідентичний код перевірки email у двох різних класах. Виправлення: виділення цієї логіки в єдиний сервіс-утиліту.